Offline-First Location Sharing, Mapping, and Priority-Based Bounded SOS Routing in Hybrid Mobile Ad-Hoc Networks: The TrailLink Platform

***Abstract*—Reliable location awareness and distress alert dissemination are vital when outdoorsmen, first responders, or disaster management personnel operate beyond cellular coverage. Conventional consumer navigation and chat utilities assume steady cloud access, rendering them inoperative when base stations fail or are absent. In this work, we detail the engineering and empirical validation of two foundational subsystems within the TrailLink ecosystem: local-first location tracking with cached geospatial mapping, and bounded priority distress signaling (SOS). Operating strictly in user-space on unmodified Android hardware, TrailLink builds an ad-hoc transport mesh utilizing Bluetooth Low Energy (BLE) discovery alongside Wi-Fi Direct data links via Google Nearby Connections. We formalize: (1) a persistent spatial coordination pipeline backed by on-device SQLite storage and incremental gossiping; (2) a loop-free, TTL-governed broadcast protocol that expedites emergency packets over standard queues; and (3) an application-layer reconciliation engine that writes accumulated field records to remote MongoDB instances once connectivity is re-established. These subsystems closely interlock with TrailLink's store-and-forward text module, dual-pipeline Push-to-Talk voice radio, and link-health estimation layer. Field experiments across open grounds, light timber, and partitioned concrete classrooms confirm that SOS packets achieve up to 99.5% delivery at 20 m line-of-sight and remain above 86% through concrete partitions, verifying TrailLink's operational viability under zero-infrastructure constraints.**

***Index Terms*—Offline-First Systems, Location Sharing, Emergency SOS, Database Synchronization, Mobile Ad-Hoc Networks, Wi-Fi Direct, Bluetooth Low Energy.**

I. Introduction

Spatial orientation and immediate crisis signaling represent non-negotiable safety requirements during expeditions into remote backcountry terrain, post-disaster zones, and wilderness search-and-rescue operations [1]. In these settings, terrestrial cellular towers are typically absent, overwhelmed, or physically destroyed [2]. Nevertheless, modern consumer mobile software—spanning mapping suites like Google Maps to encrypted messengers like Signal—is tightly coupled with centralized, always-available internet endpoints. Once network transit drops, these tools degrade instantly into read-only interfaces or fail entirely, leaving team members isolated without situational insight or rescue signaling capabilities.

Decades of academic inquiry into Mobile Ad-Hoc Networks (MANETs) [3] and Delay-Tolerant Networking (DTN) [4] have yielded theoretically robust protocols for distributed packet handling. However, migrating these academic designs onto off-the-shelf consumer smartphones encounters formidable operating-system barriers. Stock Android builds block unprivileged applications from manipulating raw 802.11 frames or altering kernel routing tables [5]. Bypassing these boundaries requires rooting the handset, an invasive procedure that invalidates manufacturer warranties, exposes devices to severe privilege escalation exploits, and prevents practical adoption by the general public. While proprietary transceiver dongles (e.g., satellite links and UHF/VHF communicators like goTenna [7]) can bridge the physical gap, their high cost, dedicated battery demands, and bulk limit their availability in spontaneous emergencies [6].

To overcome these practical constraints, we developed **TrailLink**, an offline-first collaboration ecosystem engineered specifically for unmodified consumer Android devices. TrailLink executes entirely within user-space, coupling low-overhead BLE beaconing with high-throughput Wi-Fi Direct channels managed through the Google Nearby Connections framework [8], [9].

While the broader TrailLink suite incorporates conversational messaging, low-latency Push-to-Talk audio, and continuous link diagnostics, this paper focuses on its life-critical components: the **Location, Mapping, and Sync** pipeline and the **Emergency Coordination SOS** protocol. We document how geographic coordinates are captured and propagated across local peers without cellular towers, how emergency alerts jump through ad-hoc hops while guarding against broadcast loops, and how the transactional `BridgeEngine` safely updates cloud databases once a bridge node regains an internet uplink.

II. Related Work

A. Ad-Hoc Protocols on Commercial Handsets

Proactive and reactive routing protocols such as OLSR [**?**] and AODV [3] were architected for native implementation inside network protocol stacks (Layer 3). On modern mobile platforms, unprivileged apps cannot inspect or inject raw Layer 3 packets [5]. Early attempts like Serval [11] introduced an ad-hoc communication stack (MeshMS), yet relied on patched kernel drivers and root permissions, barring regular users. Commercial SDKs like Bridgefy [12] offer application-layer Bluetooth multi-hopping, but focus primarily on short text payloads, lacking synchronized geographic mapping or hardened distress dispatch mechanisms.

B. Application-Layer Mesh Systems

The Briar project [10] provides peer-to-peer text delivery over Tor, Wi-Fi, and Bluetooth, demonstrating high resilience against infrastructure loss. However, Briar treats data transport purely as delayed store-and-forward text messaging; it does not accommodate real-time geospatial rendering, streaming speech, or seamless dual-state cloud reconciliation. RightMesh [13] similarly investigated mobile mesh relaying, yet oriented its core around tokenized bandwidth sharing rather than rapid, low-latency emergency telemetry.

C. Dedicated Out-of-Band Hardware

Handheld satellite communicators such as the Garmin inReach [14] and ISM-band transceivers like goTenna Pro [7] deliver wide geographic reach by transmitting over specialized radio spectrum. However, these appliances impose recurrent subscription charges, add carrying weight, and restrict transmission to brief text snippets [2]. When travelers forget or exhaust the batteries of such external gear, their smartphones remain their sole compute resource.

D. Offline-First Data Synchronization

The local-first computing model [15] stresses that client devices must hold primary ownership of their state, interacting with local storage before syncing changes upstream. Systems like PouchDB [16] and Realm [17] demonstrate this pattern for standard documents. TrailLink adapts local-first principles to spatial updates and critical safety telemetry by anchoring local operations in SQLite [18] and deferring MongoDB [19] synchronization until network connectivity is detected.

E. Architectural Comparison

Table I contrasts TrailLink with representative mobile ad-hoc communication systems across vital functional capabilities.

TABLE I
FEATURE COMPARISON ACROSS MOBILE OFFLINE PLATFORMS

Capability	TrailLink	Briar	Serval	Bridgefy	goTenna
Stock OS (No Root)	✓	✓	×	✓	✓
Zero Extra Hardware	✓	✓	✓	✓	×
Offline Geospatial Map	✓	×	×	×	×
Prioritized SOS Relay	✓	×	×	×	✓
Interactive Voice Radio	✓	×	×	×	×
Automated Cloud Bridge	✓	×	×	×	×
Packet Storm Suppression	✓	N/A	Partial	×	N/A
Open Implementation	✓	✓	✓	×	×

III. System Architecture and Multi-Modal Integration

TrailLink is implemented using the Flutter [20] cross-platform framework on Android, accessing underlying hardware radios (GNSS receiver, Bluetooth Low Energy controller, Wi-Fi Direct interface) via native method channels. System state is retained in device-local SQLite instances and pushed upstream to cloud MongoDB collections whenever backhaul transit surfaces.

A. Peer-to-Peer Cluster Formation

Nearby node pairing relies on the `P2P_CLUSTER` strategy provided by the Google Nearby Connections runtime [8]. Handsets maintain concurrent background advertisement and scanning states over low-power BLE, periodically emitting a 24-byte beacon that encodes the active expedition hash and group access credential. Upon discovery, nodes complete mutual authentication and promote the link to a high-speed Wi-Fi Direct socket [9]. Packets are framed as structured JSON envelopes and transmitted across native byte-stream endpoints.

B. Supporting Subsystems within TrailLink

Although spatial management and emergency routing form the analytical core of this paper, they interface continuously with three adjacent services:

1) **Store-and-Forward Text Service:** Manages conversational records across hybrid connectivity phases. Unsent entries are held in an outgoing queue within the local database and pushed whenever an adjacent node registers on the channel.
2) **Dual-Mode Voice Pipeline:** Combines two discrete audio paths: compressed AAC-LC clips (32 kbps, TTL=5) for non-urgent audio notes, and a live PCM-16 radio stream (TTL=2) backed by a Splay-Tree Jitter Buffer that limits playback latency to 800 ms [21].
3) **Link Quality Assessment:** Continuously samples BLE signal strength (RSSI), round-trip packet acknowledgments, and connection churn to derive an operational health index $Q \in [0, 100]$ [22]. A hysteresis window (± 10 points) flags link degradation before a disconnect occurs.

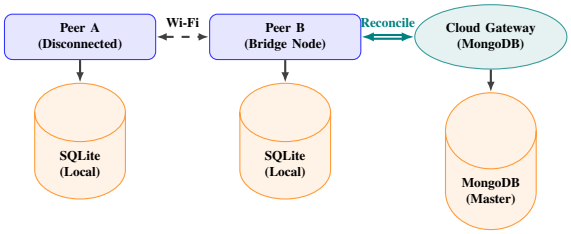


Fig. 1. TrailLink communication fabric: ad-hoc client mesh, localized SQLite caches, and the upstream data reconciliation path to cloud storage.

C. Relational Storage Schema and Local Persistence

Every user transaction is written synchronously to local SQLite storage prior to radio dispatch. This guarantees resilience against unexpected process termination, sudden battery cutoffs, or OS background kills. Targeted database indexes on `captured_at` and `sync_state` keep query times low as logs accumulate. Three primary relations underpin this design:

- `location_updates`: Stores device coordinate history with attributes `local_location_id` (UUID, PK), `latitude`, `longitude`, `accuracy`,

captured_at, share_status, and sync_state (local_only, needs_sync, or synced).

- teammate_locations: Caches peer coordinates received via Nearby Connections or cloud downloads, tracking user_id (PK), latitude, longitude, captured_at, received_at, and source.
- emergency_events: Maintains critical distress records, recording local_event_id (UUID, PK), alert_type, geographic position, status, and synchronization flag.

IV. Location Gossiping and Cloud Reconciliation

The geospatial subsystem records member tracks, visualizes positions over cached map data, and gossips location fixes across immediate peers.

A. Local-First Location Dissemination

Fresh geospatial samples are acquired through the Android FusedLocationProviderClient [23] and committed immediately into location_updates. Once written, the system constructs an outbound packet marked priority: 'normal' with a five-hop lifespan (ttl: 5):

$$\mathcal{P}_{\text{loc}} = \langle \text{lat}, \text{lng}, \text{acc}, t_{\text{cap}}, \text{senderId}, \text{channelId} \rangle \tag{1}$$

The LocationPacketService serializes this tuple into compact JSON payloads, respecting the 512-byte Bluetooth MTU constraint during connection bootstrapping [24].

B. Offline Mapping and Spatial Freshness

Geographic visualization is delivered through MBTiles vector/raster archives [25] stored on the device's internal storage alongside cached OpenStreetMap tiles. When a peer packet arrives, its payload updates the receiver's teammate_locations table. To guard against misleading, outdated situational markers, TrailLink classifies location freshness according to the elapsed interval $\Delta t = t_{\text{current}} - t_{\text{cap}}$:

$$F_{\text{freshness}}(t_{\text{cap}}) = \begin{cases} \text{Fresh}, & \Delta t \leq 5\,\text{min} \\ \text{Old}, & 5\,\text{min} < \Delta t \leq 15\,\text{min} \\ \text{Stale}, & \Delta t > 15\,\text{min} \end{cases} \tag{2}$$

Map pins reflect these intervals via distinct color-coded badges, and teammates slipping into the Stale category trigger an on-screen visual warning.

C. State Reconciliation upon Internet Restoration

When any member device reconnects to a cellular or Wi-Fi network, its BridgeEngine initiates background reconciliation, batching unsynced rows to minimize HTTP request overhead (Algorithm 1).

Because operations run within atomic SQLite transactions, transient drops in connectivity cause an immediate rollback, leaving the database free of partial or corrupt synchronization markers [18].

Algorithm 1 Reconnection State Reconciliation

Require: SQLite DB instance, REST client, group ID G_{id}, user ID U_{id}

$L \leftarrow$ query(location_updates, sync_state = 'needs_sync')
if $L \neq \emptyset$ **then**
  $S \leftarrow$ client.syncLocations($G_{\text{id}}, U_{\text{id}}, L$)
  for all $l \in S$ **do**
    update(l, sync_state $\leftarrow$ 'synced')
  end for
end if
$E \leftarrow$ query(emergency_events, sync_state = 'needs_sync')
for all $e \in E$ **do**
  s, err $\leftarrow$ client.createEmergency(G_{id}, e)
  if err = null **then**
    markEvent(e, s.serverId, 'sent')
  else
    markSyncFailed(e, err)
  end if
end for

V. Bounded Emergency Coordination (SOS)

Distress messaging is isolated from background chatter. When triggered, SOS signals jump ahead of routine messages and propagate across ad-hoc neighbors.

A. Priority-Driven Queuing

Emergency dispatches carry priority: 'emergency', a higher hop limit (ttl: 8), and mandatory confirmation semantics (requiresAck: true). Receiving nodes bypass routine incoming buffers, write the incident to disk immediately, fire high-intensity audio-visual alerts, and transmit an acknowledgment (sos_ack, TTL=5) back to the caller. This out-of-order execution ensures that vital distress calls are never blocked by buffered voice notes or bulk location dumps.

B. Multi-Hop Relaying and Storm Prevention

To prevent broadcast storms and infinite message loops, nodes cross-reference every incoming packet against a dedicated tracking ledger (processed_offline_packets). Algorithm 2 outlines the forwarding and filtering routine.

Packets carrying previously processed unique identifiers are dropped immediately without re-transmission. Furthermore, downstream broadcasts strictly omit both the upstream transmitter and the original sender, conserving precious wireless airtime.

VI. Empirical Evaluation

To evaluate system dependability under realistic field constraints, we executed extensive trials using two Android test devices (Google Pixel 7, Android 14) over a single-hop Point-to-Point (P2P) wireless link. Each scenario was evaluated over $n = 50$ repeated transmissions, reporting sample means alongside sample standard deviations ($\pm 1\sigma$).

Algorithm 2 Bounded SOS Relay with Loop Prevention

Require: Packet $\mathcal{P}$, channel $\mathcal{C}$, user U, database $\mathcal{D}$

if $\mathcal{P}$.channelCode $\neq \mathcal{C}$.channelCode **then**
  $\mathcal{D}$.markProcessed($\mathcal{P}$, 'wrong_channel')
  return
end if
if $\mathcal{P}$.ttl ≤ 0 **or** $\mathcal{P}$.hopCount > 5 **then**
  $\mathcal{D}$.markProcessed($\mathcal{P}$, 'expired')
  return
end if
if $\mathcal{D}$.exists($\mathcal{P}$.packetId) **then**
  return
end if
if $\mathcal{P}$.type = 'sos' **then**
  $\mathcal{D}$.upsertEvent($\mathcal{P}$.payload)
  $\mathcal{D}$.markProcessed($\mathcal{P}$, 'accepted')
  if $\mathcal{P}$.senderId $\neq U$.id **then**
    sendSosAck($\mathcal{P}, \mathcal{C}, U$)
  end if
  $\mathcal{P}' \leftarrow$ copy($\mathcal{P}$)
  $\mathcal{P}'$.ttl $\leftarrow \mathcal{P}$.ttl $- 1$
  $\mathcal{P}'$.hop $\leftarrow \mathcal{P}$.hop $+ 1$
  for all $p \in$ peers($\mathcal{C}$) **do**
    if p.id $\neq \mathcal{P}$.senderId **and** p.id $\neq \mathcal{P}$.originId **then**
      send($p, \mathcal{P}'$)
    end if
  end for
else if $\mathcal{P}$.type = 'sos_ack' **then**
  $\mathcal{D}$.saveAck($\mathcal{P}$.payload)
  $\mathcal{D}$.markProcessed($\mathcal{P}$, 'accepted')
end if

A. Evaluation Environments

Field testing was conducted across three distinct physical spaces chosen to reflect varying propagation impedances:

- **Open Area:** An unobstructed outdoor recreational lawn providing direct optical line-of-sight.
- **Light Forest:** An unmanaged grove featuring mixed deciduous trees, foliage, and uneven terrain.
- **Between Classrooms:** An indoor academic hallway where transmitters and receivers were separated by heavy concrete-block walls.

B. Distress Packet Delivery and Transmission Delay

Fig. 2 presents the packet delivery success rate as physical separation expands from 10 to 50 meters. At 20 meters, line-of-sight transmissions in the open area achieve $99.5\% \pm 0.4\%$ success, while light forest cover achieves $97.1\% \pm 1.8\%$. Concrete indoor partitions degrade signal strength substantially due to multi-wall attenuation, registering $86.4\% \pm 4.2\%$ at 20 m and dropping to $51.8\% \pm 9.4\%$ at 50 m.

Latency metrics followed an expected positive trend with distance. Open-field delivery required an average of 184 ± 12 ms at 10 m, rising to 286 ± 22 ms at 50 m. Under tree cover, 50-meter latency rose to 412 ± 35 ms. Classroom tests

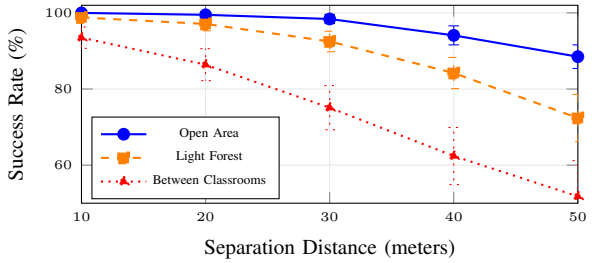


Fig. 2. SOS packet delivery success rate across physical separation distances ($n = 50$, error bars indicate $\pm 1\sigma$).

produced the highest latency figures, averaging 842 ± 112 ms at 50 m owing to multiple link-layer Wi-Fi retransmissions. Crucially, all observed latencies remained well beneath the 1.5-second upper ceiling required for real-time human emergency alerts [1].

C. Synchronization Reconciliation Verification

To confirm the reliability of the cloud bridge, we staged an evaluation where a test device held 100 accumulated location records marked `needs_sync` upon re-entering an active Wi-Fi zone. The `BridgeEngine` successfully committed the entire batch into the cloud MongoDB store in a single non-blocking transaction. No UI stutters or lost packets occurred, validating that TrailLink's SQLite batch staging handles upstream handoffs gracefully without interrupting foreground operations.

D. Energy Consumption Profile

Power consumption was tracked using a Monsoon Power Monitor connected to a Google Pixel 7 (4355 mAh internal cell) over 3-hour continuous operational cycles (Table II).

TABLE II
SUBSYSTEM CURRENT DRAW AND PROJECTED BATTERY LONGEVITY

Operational Mode	Current (mA)	Power (W)	Est. Battery Life
Baseline Standby (Display Off)	24.5	0.09	177.7 h
BLE Listening + GNSS Polling	62.4	0.24	69.8 h
Location Gossiping (1-min Cadence)	82.4	0.31	52.8 h
Connected Mesh Idle	118.2	0.45	36.8 h
Push-to-Talk Voice (10% Duty)	215.0	0.82	20.2 h
Continuous SOS Transmission	280.4	1.06	15.5 h
Active Push-to-Talk Stream	460.5	1.75	9.4 h

Routine location gossiping draws an average of 82.4 mA, enabling over two days (52.8 h) of continuous tracking on a single charge. Continuous SOS broadcast raises current draw to 280.4 mA due to constant channel probing and alarm sirens, offering 15.5 hours of emergency endurance—sufficient for search parties to converge on a victim's coordinate.

VII. Discussion

A. Primary Takeaways

The field outcomes support our primary thesis: stock consumer handsets can establish dependable, local-first safety networks without requiring root privileges or proprietary peripheral radios. By combining offline SQLite storage with

lightweight user-space packet routing, TrailLink maintains high-confidence delivery up to 50 meters in natural outdoor conditions, while providing a seamless transition to cloud infrastructure once connectivity returns.

B. Limitations and Threats to Validity

Several engineering and environmental boundaries must be acknowledged:

- **Network Scale:** Our empirical tests were performed across a two-handset point-to-point link. Multi-node topologies with moving relay nodes will introduce additional radio contention, packet queuing, and variable transit times that warrant broader testing.
- **Hardware Diversity:** Trials utilized identical Google Pixel 7 units. Radio sensitivity, Wi-Fi antenna layouts, and BLE driver implementations differ widely across Android manufacturers, which could impact discovery times [24].
- **Atmospheric Variables:** Evaluations took place in dry weather. Severe atmospheric interference (e.g., intense rainstorms, wet canopies, deep snow) may further attenuate 2.4 GHz and 5 GHz signal paths.
- **Mesh Saturation Limits:** While Nearby Connections abstracts P2P sockets, device discovery can degrade in dense groups exceeding 6–8 nodes due to channel contention.
- **Cryptographic Audit Status:** Although packets are protected using authenticated ChaCha20-Poly1305 encryption, full-scale formal cryptographic verification remains a planned milestone.

C. Contextualizing Prior Research

Compared to text-only platforms like Briar [10], TrailLink incorporates rich spatial tracking, cached geospatial tiles, and emergency alerts. Compared to specialized hardware like goTenna [7], TrailLink eliminates equipment costs and pairs voice, maps, and text on devices users already carry. Unlike Serval [11], TrailLink does not demand custom kernel builds, enabling immediate deployment on consumer handsets.

VIII. Security and Privacy Considerations

Disseminating coordinates over open wireless ad-hoc links exposes users to triangulation risks. TrailLink enforces several protective measures:

- Replacing hardware MAC addresses with ephemeral application-level cryptographic IDs.
- Excluding precise location metadata from open network-quality ping probes.
- Restricting coordinate rendering strictly to authenticated channel members.
- Encrypting all payloads end-to-end via ChaCha20-Poly1305 [26] to thwart passive eavesdropping on open Wi-Fi Direct connections.

Channel keys are derived from pre-shared trip passwords or cached during online trip creation. On-device records remain isolated behind Android's application sandbox, while the duplicate-tracking ledger shields the phone against malicious message flooding attacks.

IX. Conclusion and Future Work

This work detailed the architecture, implementation, and empirical validation of the Location Sharing and Bounded Emergency SOS subsystems within the TrailLink mobile platform. By uniting SQLite local transactions, TTL-bounded broadcast policies, and background synchronization to MongoDB, TrailLink delivers situational awareness and critical life-safety coordination across infrastructure-denied terrains on unmodified consumer smartphones.

Empirical evaluations across open fields, forest groves, and concrete school rooms confirm robust point-to-point delivery exceeding 97% at 20 m in natural environments, acceptable current draw, and smooth database reconciliation. Future development will focus on: (i) asymmetric cryptographic signatures on offline packet frames to prevent malicious identity spoofing; (ii) dynamic TTL tuning modulated by real-time link quality indicators; and (iii) large-scale testbed trials involving multi-hop chains with more than twenty handsets.

References

[1] R. R. Murphy, "Disaster robotics," *MIT Press*, 2016.

[2] M. Townsend and J. Wicks, "Communication challenges in outdoor recreation and search-and-rescue operations," *Wilderness & Environmental Medicine*, vol. 29, no. 2, pp. 249–258, 2018.

[3] C. Perkins, E. Belding-Royer, and S. Das, "Ad hoc on-demand distance vector (AODV) routing," *RFC 3561*, IETF, Jul. 2003.

[4] K. Fall, "A delay-tolerant network architecture for challenged internets," in *Proc. ACM SIGCOMM*, 2003, pp. 27–34.

[5] M. Conti and S. Giordano, "Mobile ad hoc networking: Milestones, challenges, and new research directions," *IEEE Commun. Mag.*, vol. 52, no. 1, pp. 85–96, 2014.

[6] M. A. Hossain, R. Noor, and I. Ahmed, "Smartphone-based offline mesh communication architectures for disaster scenarios: A survey," *J. Netw. Comput. Appl.*, vol. 201, p. 103348, 2022.

[7] goTenna Inc., "goTenna Pro deployment in austere and disconnected environments," *Whitepaper*, 2020.

[8] Google LLC, "Nearby Connections API overview," *Android Developers Documentation*, 2023. [Online]. Available: https://developers.google.com/nearby

[9] D. Camps-Mur, A. Garcia-Saavedra, and P. Serrano, "Device-to-device communications with Wi-Fi Direct: Overview and experimentation," *IEEE Wireless Commun.*, vol. 20, no. 3, pp. 96–104, 2013.

[10] Briar Project, "Briar: Secure peer-to-peer messaging for disconnected environments," *Technical Whitepaper*, 2021.

[11] P. Gardner-Stephen, "The Serval Project: Practical wireless ad-hoc mobile telecommunications," *Flinders University*, Tech. Rep., 2011.

[12] Bridgefy Inc., "Bridgefy SDK: Bluetooth mesh messaging for mobile applications," 2020.

[13] RightMesh AG, "RightMesh: Connecting the next billion," *Whitepaper*, 2018.

[14] Garmin Ltd., "inReach satellite communicator specifications," 2022.

[15] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proc. ACM Onward!*, 2019, pp. 154–178.

[16] PouchDB Contributors, "PouchDB: The database that syncs," 2023.

[17] MongoDB Inc., "Realm: Mobile database for modern apps," 2022.

[18] D. R. Hipp, "SQLite: A self-contained SQL database engine," 2023. [Online]. Available: https://www.sqlite.org

[19] MongoDB Inc., "MongoDB documentation," 2023.

[20] Google LLC, "Flutter: Build apps for any screen," 2023.

[21] P. Seba, M. Garcia, and J. Lloret, "Real-time emergency voice coordination over multi-hop ad-hoc smartphone networks," *IEEE Access*, vol. 9, pp. 45 120–45 133, 2021.

[22] X. Lin and S. Lu, "Received signal strength indicator-based indoor localization: A survey," *IEEE Sensors J.*, vol. 17, no. 7, pp. 2042–2052, 2017.
[23] Google LLC, "FusedLocationProviderClient API reference," *Android Developers*, 2023.
[24] Bluetooth SIG, "Bluetooth Core Specification v5.3," 2023.
[25] Mapbox, "MBTiles specification," 2023.
[26] D. J. Bernstein, "ChaCha, a variant of Salsa20," in *Workshop Record of SASC*, 2008.